



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
"МИРЭА — Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №8
по дисциплине
«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Боргачев Т. М.

Принял доцент кафедры ЦТ

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №8. СЛОЖНЫЕ ТИПЫ ДАННЫХ И МАСШТАБИРОВАНИЕ POSTGRESQL

Цель: освоение методов работы со сложными и неструктурированными типами данных (JSONB, BYTEA) в PostgreSQL, а также получение практических навыков реализации горизонтального масштабирования базы данных с помощью декларативного секционирования.

Постановка задачи:

Задание №1: хранение паролей (BYTEA)

1. Создать справочную таблицу ролей и таблицу системных пользователей (или модифицировать существующую).
2. В таблице пользователей предусмотреть поля для хранения «сырого» пароля (только для учебной демонстрации) и его хеша (тип BYTEA).
3. Сохранить хеши паролей пользователей, используя функцию хеширования digest.
4. Выполнить запрос, проверяющий соответствие введенного пароля сохраненному хешу. Одна проверка должна быть успешной, другая - нет (допустимо сделать это в одном запросе).

Задание №2: секционирование таблицы логов

1. Создать новую секционированную таблицу для хранения логов событий.
2. Использовать стратегию секционирования по диапазонам (RANGE) по полю даты.
3. Создать две секции для периодов:
2-е полугодие 2024 (с 2024-07-01 по 2025-01-01).
1-е полугодие 2025 (с 2025-01-01 по 2025-07-01).
4. Создать секцию по умолчанию, куда будут попадать все данные, выходящие за рамки указанных диапазонов (будущее время).

Задание №3: генерация данных

Написать скрипт на PL/pgSQL для генерации 20 000 записей в диапазоне дат, покрывающем созданные секции. Сгенерированные данные должны содержать JSON-структуру с различным набором полей для разных типов событий (например, чтобы логи «продажи» отличались по структуре от логов «входа»). Вывести сгенерированные данные – показать запросом SELECT, что данные успешно созданы и физически распределены по разным таблицам-секциям.

Задание №4: анализ и поиск по JSONB

Написать три запроса:

1. Фильтрация по значению – найти записи, где числовое поле внутри JSON

удовлетворяет математическому условию (например, > 800), используя операторы извлечения $->$ и $->>$.

2. Поиск по вхождению – найти записи, содержащие точный фрагмент JSON-структуры (например, `{"quantity": 5}`), используя оператор $@>$.

3. Агрегация – посчитать сумму или среднее значение числового поля из JSON-документа.

Задание №5: модификация данных JSONB

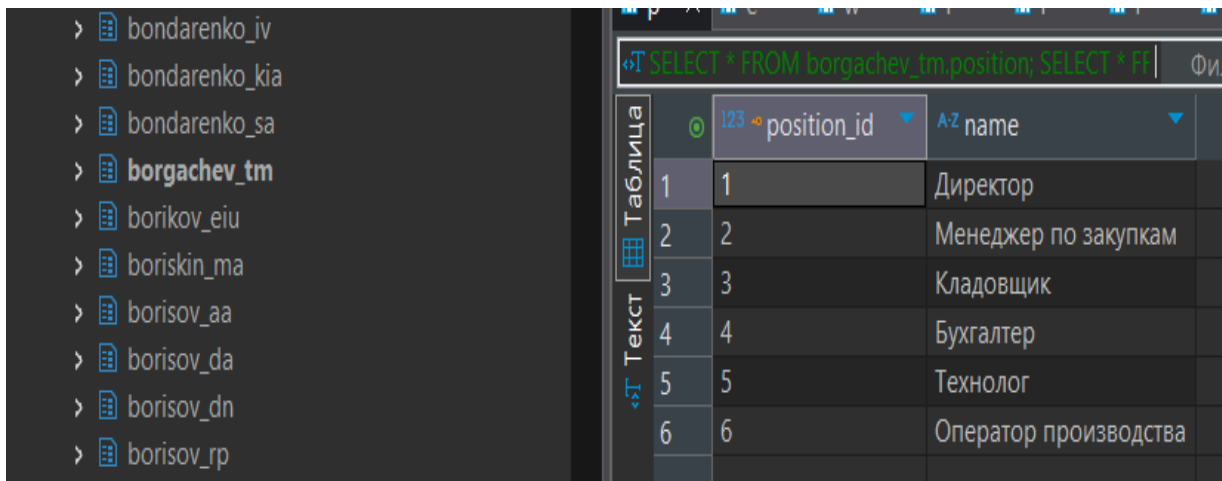
Продемонстрировать изменение данных внутри JSON-поля:

1. Массовое добавление нового поля во все записи определенного типа.
2. Точечное изменение значения по ключу в конкретной записи.

ХОД РАБОТЫ

Задание №1: хранение паролей (BYTEA).

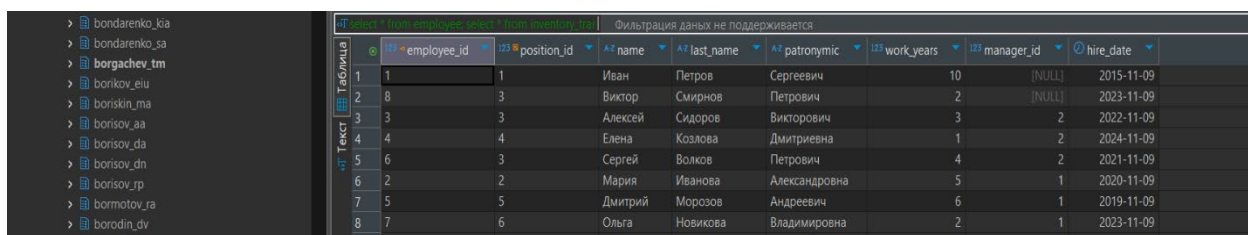
Таблица position представлена на Рисунке 1.



position_id	name
1	Директор
2	Менеджер по закупкам
3	Кладовщик
4	Бухгалтер
5	Технолог
6	Оператор производства

Рисунок 1 —Таблица position

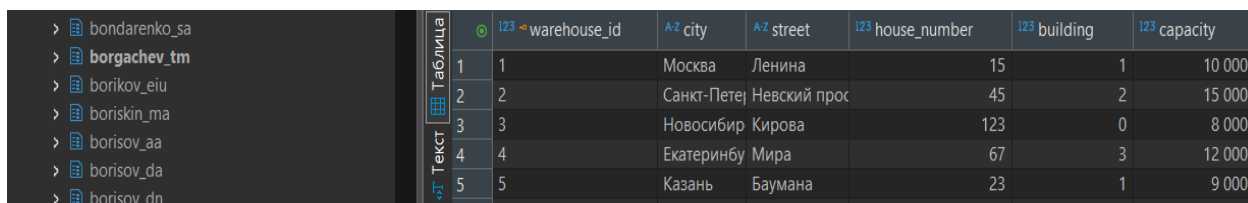
Таблица employee представлена на Рисунке 2.



employee_id	position_id	name	last_name	patronymic	work_years	manager_id	hire_date
1	1	Иван Петров Сергеевич	Иван	Петров	10	[NULL]	2015-11-09
2	8	Виктор Смирнов Петрович	Виктор	Смирнов	2	[NULL]	2023-11-09
3	3	Алексей Сидоров Викторovich	Алексей	Сидоров	3	2	2022-11-09
4	4	Елена Козлова Дмитриевна	Елена	Козлова	1	2	2024-11-09
5	6	Сергей Волков Петрович	Сергей	Волков	4	2	2021-11-09
6	2	Мария Иванова Александровна	Мария	Иванова	5	1	2020-11-09
7	5	Дмитрий Морозов Андреевич	Дмитрий	Морозов	6	1	2019-11-09
8	7	Ольга Новикова Владимировна	Ольга	Новикова	2	1	2023-11-09

Рисунок 2 —Таблица employee

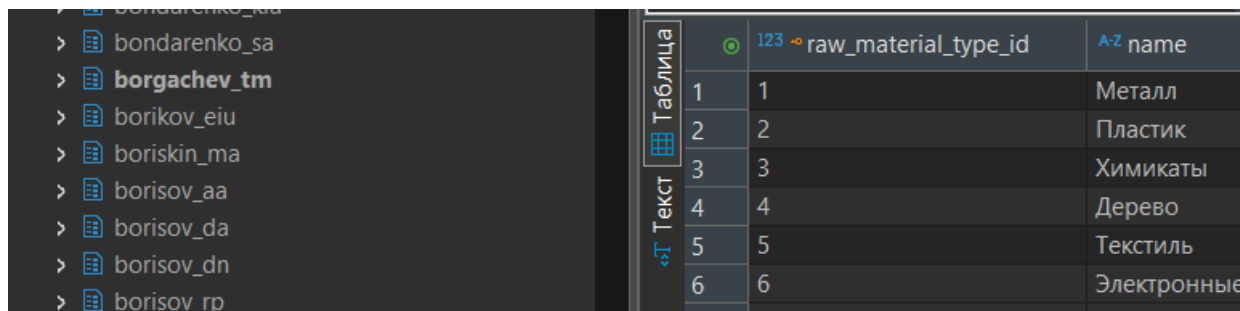
Таблица warehouse представлена на Рисунке 3.



warehouse_id	city	street	house_number	building	capacity
1	Москва	Ленина	15	1	10 000
2	Санкт-Петербург	Невский прот	45	2	15 000
3	Новосибирск	Кирова	123	0	8 000
4	Екатеринбург	Мира	67	3	12 000
5	Казань	Баумана	23	1	9 000

Рисунок 3 —Таблица warehouse

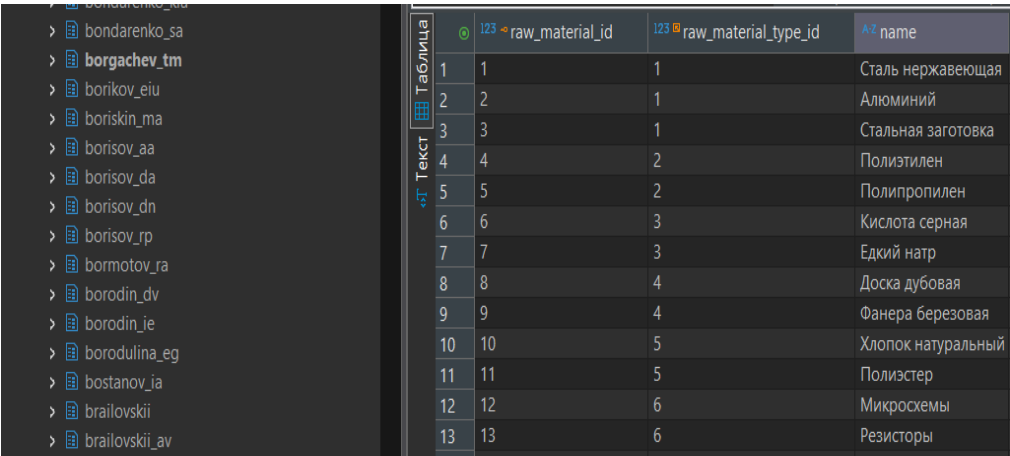
Таблица raw_material_type представлена на Рисунке 4.



raw_material_type_id	name
1	Металл
2	Пластик
3	Химикаты
4	Дерево
5	Текстиль
6	Электронные

Рисунок 4 —Таблица raw_material_type

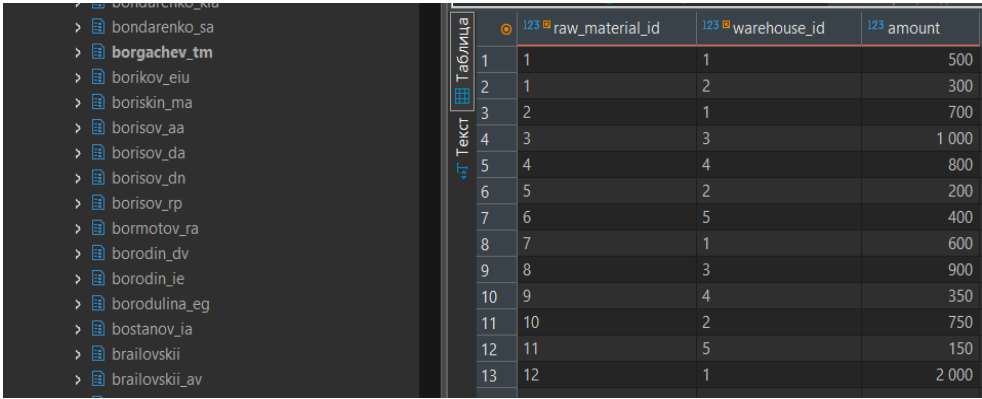
Таблица raw_material представлена на Рисунке 5.



	123 raw_material_id	123 raw_material_type_id	A-Z name
1	1	1	Сталь нержавеющая
2	2	1	Алюминий
3	3	1	Стальная заготовка
4	4	2	Полиэтилен
5	5	2	Полипропилен
6	6	3	Кислота серная
7	7	3	Едкий натр
8	8	4	Доска дубовая
9	9	4	Фанера березовая
10	10	5	Хлопок натуральный
11	11	5	Полиэстер
12	12	6	Микросхемы
13	13	6	Резисторы

Рисунок 5 —Таблица raw_material

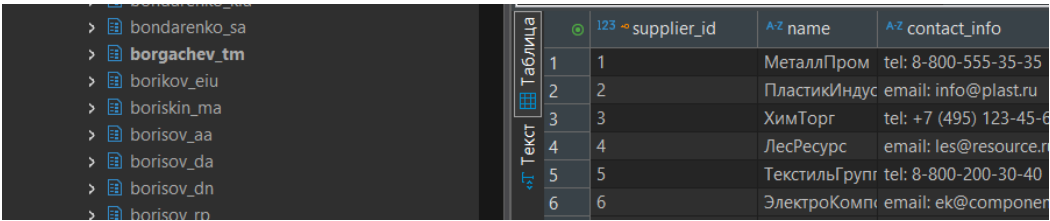
Таблица inventory представлена на Рисунке 6.



	123 raw_material_id	123 warehouse_id	123 amount
1	1	1	500
2	1	2	300
3	2	1	700
4	3	3	1 000
5	4	4	800
6	5	2	200
7	6	5	400
8	7	1	600
9	8	3	900
10	9	4	350
11	10	2	750
12	11	5	150
13	12	1	2 000

Рисунок 6 —Таблица inventory

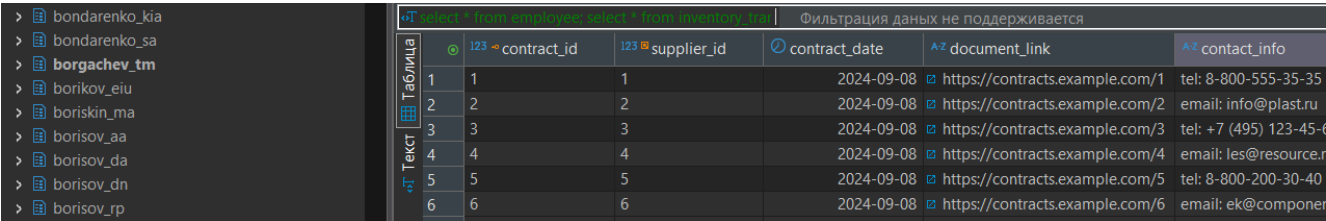
Таблица supplier представлена на Рисунке 7.



	123 supplier_id	A-Z name	A-Z contact_info
1	1	МеталлПром	tel: 8-800-555-35-35
2	2	ПластикИндус	email: info@plast.ru
3	3	ХимТорг	tel: +7 (495) 123-45-6
4	4	ЛесРесурс	email: les@resource.ru
5	5	ТекстильГрупп	tel: 8-800-200-30-40
6	6	ЭлектроКомп	email: ek@componen

Рисунок 7 —Таблица supplier

Таблица contract представлена на Рисунке 8.



	123 contract_id	123 supplier_id	contract_date	A-Z document_link	A-Z contact_info
1	1	1	2024-09-08	https://contracts.example.com/1	tel: 8-800-555-35-35
2	2	2	2024-09-08	https://contracts.example.com/2	email: info@plast.ru
3	3	3	2024-09-08	https://contracts.example.com/3	tel: +7 (495) 123-45-6
4	4	4	2024-09-08	https://contracts.example.com/4	email: les@resource.ru
5	5	5	2024-09-08	https://contracts.example.com/5	tel: 8-800-200-30-40
6	6	6	2024-09-08	https://contracts.example.com/6	email: ek@componen

Рисунок 8 —Таблица contract

Таблица delivery_act представлена на Рисунке 9.

	act_id	warehouse_id	delivery_date
1	1	1	2025-09-03
2	2	2	2025-09-04
3	3	3	2025-09-05
4	4	4	2025-09-06
5	5	5	2025-09-07

Рисунок 9 —Таблица delivery_act

Таблица material_order представлена на Рисунке 10.

	order_id	raw_material_id	employee_id	supplier_id	contract_id	order_date
1	1	1	3	1	1	2025-09-03 1
2	2	2	3	1	1	2025-09-03 1
3	3	1	3	1	1	2025-09-03 1
4	4	2	3	1	1	2025-09-03 1
5	5	1	3	1	1	2025-09-03 1
6	6	2	3	1	1	2025-09-03 1
7	7	1	3	1	1	2025-09-07 5
8	8	1	3	1	1	2025-09-07 5
9	9	1	3	1	1	2025-09-07 5
10	10	1	3	1	1	2025-09-07 5
11	11	1	3	1	1	2025-09-07 5
12	34	1	3	1	1	2024-01-15 1
13	35	2	3	2	2	2024-01-20 2
14	36	3	2	3	3	2024-02-05 3
15	37	4	3	4	4	2024-02-10 4
16	38	5	2	5	5	2024-02-15 5
17	39	6	3	1	1	2024-03-01 1
18	40	7	2	2	2	2024-03-05 2
19	41	8	3	3	3	2024-03-10 3
20	42	9	2	4	4	2024-03-15 4
21	43	10	3	5	5	2024-04-01 5
22	44	11	2	1	1	2024-04-05 1
23	45	12	3	2	2	2024-04-10 2

Рисунок 10 —Таблица material_order

Таблица detail_type представлена на Рисунке 11.

	detail_type_id	name
1	1	Металл
2	2	Пластик
3	3	Химикаты
4	4	Дерево
5	5	Текстиль
6	6	Электронные
7	7	Турбина
8	8	Сталь

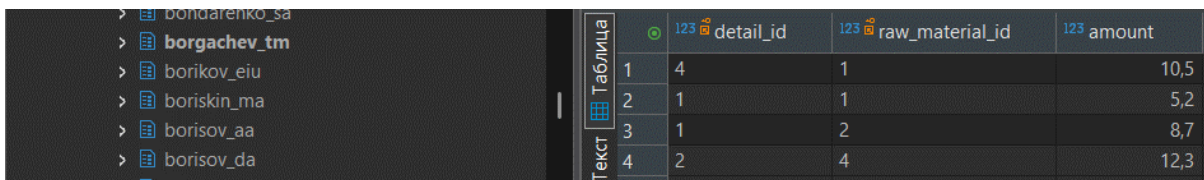
Рисунок 11 —Таблица detail_type

Таблица detail представлена на Рисунке 12.

	detail_id	detail_type_id	name
1	1	1	Деталь 1
2	2	2	Деталь 2
3	3	7	Турбина ГТД
4	4	8	Стальная деталь

Рисунок 12 —Таблица detail

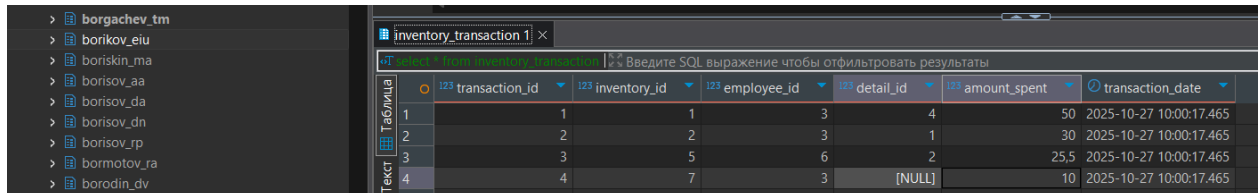
Таблица detail_material представлена на Рисунке 13.



	123 detail_id	123 raw_material_id	123 amount
1	4	1	10,5
2	1	1	5,2
3	1	2	8,7
4	2	4	12,3

Рисунок 13 — Таблица detail_material

Таблица inventory_transaction представлена на Рисунке 14.



	123 transaction_id	123 inventory_id	123 employee_id	123 detail_id	123 amount_spent	transaction_date
1	1	1	3	4	50	2025-10-27 10:00:17.465
2	2	2	3	1	30	2025-10-27 10:00:17.465
3	3	5	6	2	25,5	2025-10-27 10:00:17.465
4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465

Рисунок 14 — Таблица inventory_transaction

Таблица inventory_transaction_audit представлена на Рисунке 15.



	123 audit_id	123 transaction_id	123 inventory_id	123 employee_id	123 detail_id	123 amount_spent	transaction_date	operation	audit_timestamp
1	1	1	1	3	4	50	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
2	2	2	2	3	1	30	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
3	3	3	5	6	2	25,5	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
4	4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
5	5	5	1	[NULL]	[NULL]	100	2025-10-27 10:07:52.704	HISTORICAL	2025-11-09 16:45:28.556

Рисунок 15 — Таблица inventory_transaction_audit

Создание таблицы с паролями пользователей представлено в Листинге 1.

Листинг 1 — Создание таблиц с ролями и паролями

```
-- Справочник ролей
CREATE TABLE IF NOT EXISTS borgachev_tm.roles (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE
);

-- Таблица системных пользователей
CREATE TABLE IF NOT EXISTS borgachev_tm.system_users (
    user_id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    role_id INT NOT NULL REFERENCES borgachev_tm.roles(role_id),
    password_raw TEXT,
    password_hash BYTEA NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Добавление ролей
INSERT INTO borgachev_tm.roles (role_name)
VALUES ('admin'), ('manager'), ('user'), ('auditor')
ON CONFLICT DO NOTHING;

-- Добавление пользователей с хешированием
INSERT INTO borgachev_tm.system_users (username, role_id, password_raw, password_hash)
VALUES
('admin', 1, 'supersecret', digest('supersecret', 'sha256')),
('manager_ivan', 2, 'ivPass123', digest('ivPass123', 'sha256')),
('user_kate', 3, 'kate2025', digest('kate2025', 'sha256')),
('auditor_alex', 4, 'audit!', digest('audit!', 'sha256')),
('user', 3, 'password', digest('password!', 'sha256'));

SELECT
    username,
    password_raw,
    encode(password_hash, 'hex') AS password_hash,
    (digest(password_raw, 'sha256') = password_hash) as is_valid_check
FROM system_users;
```

Результат создания представлен на Рисунке 16.

	username	password_raw	password_hash	is_valid_check
1	admin	supersecret	f75778f7425be4db0369d09af37a6c2b9a83dea0e53e7bd57412e4b060e607f7	[v]
2	manager_jvan	ivPass123	094e969342781138efee057d45b3317ac182f753ba359a6bc76adee3a22a2268	[v]
3	user_kate	kate2025	518611e3e0b36fc4b750e286ebc7f62c50735f60a27bbca3228eb17232a69364	[v]
4	auditor_alex	audit!	a98595626a905e79bfd4d86c5d28c2dc5fad0c6e6beed0461005bfc9d66e038c	[v]
5	user	password	f82a7d02e8f0a728b7c3e958c278745cb224d3d7b2e3b84c0ecafc5511fdbdb7	[]

Рисунок 16 — Таблица с паролями

Задание №2: секционирование таблицы логов.

Скрипт для создания и секционирования таблицы логов представлен в Листинге 2.

Листинг 2 — Секционирование логов

```
-- Задание №2
CREATE TABLE borgachev_tm.user_logs (
    log_id BIGSERIAL NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    user_id INT,
    event_data JSONB NOT NULL
) PARTITION BY RANGE (created_at);

-- Секция 1: 2-е полугодие 2024 (01.07.2024 - 01.01.2025)
CREATE TABLE borgachev_tm.user_logs_2024_h2
PARTITION OF borgachev_tm.user_logs
FOR VALUES FROM ('2024-07-01') TO ('2025-01-01');

-- Секция 2: 1-е полугодие 2025 (01.01.2025 - 01.07.2025)
CREATE TABLE borgachev_tm.user_logs_2025_h1
PARTITION OF borgachev_tm.user_logs
FOR VALUES FROM ('2025-01-01') TO ('2025-07-01');

-- Секция по умолчанию
CREATE TABLE borgachev_tm.user_logs_default
PARTITION OF borgachev_tm.user_logs
DEFAULT;

CREATE INDEX idx_user_logs_date ON borgachev_tm.user_logs(created_at);
```

Результат представлен на Рисунке 17.

▼	borgachev_tm	
▼	Таблицы	
>	contract	48K
>	delivery_act	40K
>	detail	40K
>	detail_material	32K
>	detail_type	40K
>	employee	24K
>	inventory	40K
>	inventory_detail	8K
>	inventory_transaction	24K
>	inventory_transaction_audit	32K
>	material_order	4,2M
>	position	24K
>	production_act	8K
>	raw_material	40K
>	raw_material_type	24K
>	roles	40K
>	supplier	40K
>	system_users	48K
>	transfer_act	8K
>	transfer_detail	8K
>	transfer_material	8K
▼	user_logs	48K
>	Колонки	
>	Ограничения	
>	Внешние ключи	
>	Индексы	
>	Зависимости	
>	Ссылки	
▼	Секции таблиц	
>	user_logs_2024_h2	16K
>	user_logs_2025_h1	16K
>	user_logs_default	16K
>	Дочерние таблицы	
>	Триггеры	
>	Правила	

Рисунок 17 — Секции таблицы user_logs

Задание №3: генерация данных.

Скрипт для генерации записей представлен в Листинге 3.

Листинг 3 — Генерация данных

```
DO $$
DECLARE
    i INT;
    random_date TIMESTAMPTZ;
    random_user INT;
    event_type TEXT;
    json_payload JSONB;
BEGIN
    FOR i IN 1..20000 LOOP
        -- Генерация даты в диапазоне 07.2024 - 07.2025
        random_date := '2024-07-01'::TIMESTAMPTZ +
            random() * ('2025-07-01'::TIMESTAMPTZ - '2024-07-
01'::TIMESTAMPTZ);

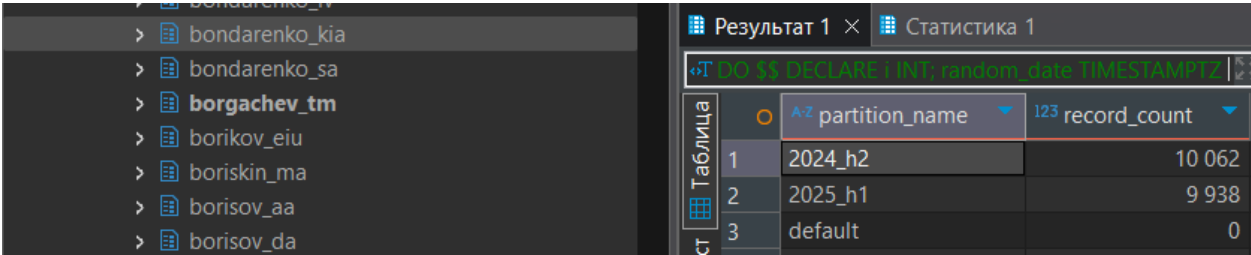
        -- Случайный пользователь
        random_user := CASE
            WHEN random() > 0.5 THEN
                (SELECT employee_id FROM borgachev_tm.employee WHERE name = 'Алексей'
LIMIT 1)
            ELSE
                (SELECT employee_id FROM borgachev_tm.employee WHERE name = 'Сергей'
LIMIT 1)
            END;

        -- Чередование типов событий
        IF (i % 2 = 0) THEN
            event_type := 'login';
            json_payload := jsonb_build_object(
                'event', event_type,
                'ip', format('192.168.%s.%s', (random() * 255)::INT, (random() *
255)::INT),
                'success', (random() > 0.1)
            );
        ELSE
            event_type := 'material_order';
            json_payload := jsonb_build_object(
                'event', event_type,
                'details', jsonb_build_object(
                    'material_id', (SELECT raw_material_id FROM borgachev_tm.raw_ma-
terial ORDER BY random() LIMIT 1),
                    'quantity', (1 + random() * 10)::INT,
                    'supplier_id', (SELECT supplier_id FROM borgachev_tm.supplier OR-
DER BY random() LIMIT 1),
                    'warehouse_id', (SELECT warehouse_id FROM borgachev_tm.warehouse
ORDER BY random() LIMIT 1)
                )
            );
        END IF;

        INSERT INTO borgachev_tm.user_logs (created_at, user_id, event_data)
        VALUES (random_date, random_user, json_payload);
    END LOOP;
END $$;

SELECT
    '2024_h2' AS partition_name,
    COUNT(*) AS record_count
FROM borgachev_tm.user_logs_2024_h2
UNION ALL
SELECT '2025_h1', COUNT(*) FROM borgachev_tm.user_logs_2025_h1
UNION ALL
SELECT 'default', COUNT(*) FROM borgachev_tm.user_logs_default;
```

Проверка результата представлена на Рисунке 18.



	partition_name	record_count
1	2024_h2	10 062
2	2025_h1	9 938
3	default	0

Рисунок 18 — Проверка генерации

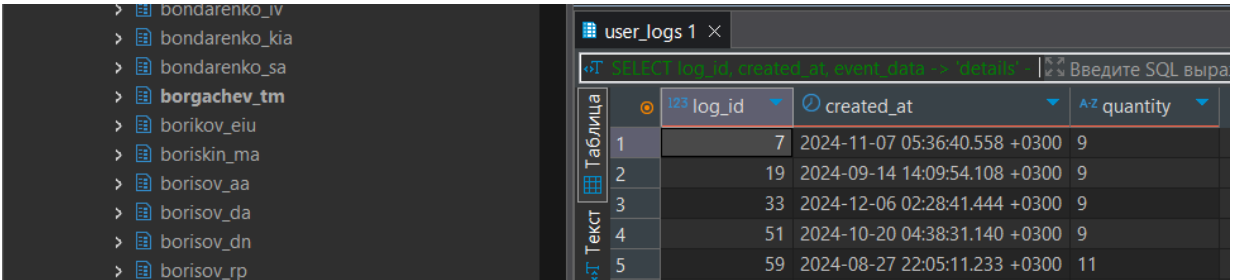
Задание №4: анализ и поиск по JSONB.

Запрос для фильтрации по значению представлен в Листинге 4.

Листинг 4 — Фильтрация по значению

```
-- 1. Фильтрация по количеству материалов в заказе (> 8 единиц)
SELECT
    log_id,
    created_at,
    event_data -> 'details' -> 'quantity' AS quantity
FROM borgachev_tm.user_logs
WHERE
    event_data -> 'event' = 'material_order'
    AND (event_data -> 'details' -> 'quantity')::INT > 8
LIMIT 5;
```

Результат запроса представлен на Рисунке 19.



	log_id	created_at	quantity
1	7	2024-11-07 05:36:40.558 +0300	9
2	19	2024-09-14 14:09:54.108 +0300	9
3	33	2024-12-06 02:28:41.444 +0300	9
4	51	2024-10-20 04:38:31.140 +0300	9
5	59	2024-08-27 22:05:11.233 +0300	11

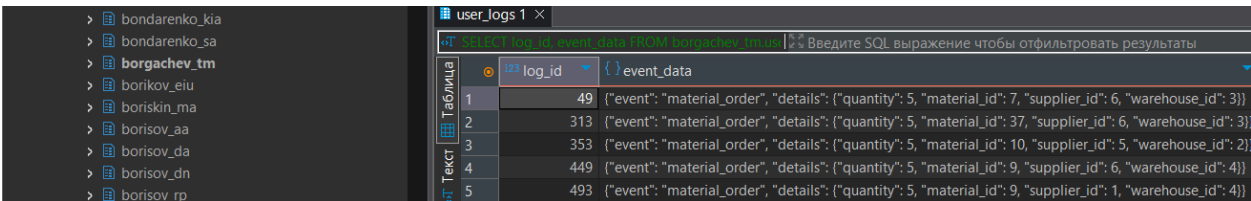
Рисунок 19 — Фильтрация по заказам сырья

Запрос для поиска по JSON представлен в Листинге 5.

Листинг 5 — Поиск по JSON-фрагменту

```
-- 2. Поиск по точному фрагменту JSON (заказы с количеством 5 единиц)
SELECT log_id, event_data
FROM borgachev_tm.user_logs
WHERE event_data @> '{"details": {"quantity": 5}}'::jsonb
LIMIT 5;
```

Результат запроса представлен на Рисунке 20.



	log_id	event_data
1	49	{"event": "material_order", "details": {"quantity": 5, "material_id": 7, "supplier_id": 6, "warehouse_id": 3}}
2	313	{"event": "material_order", "details": {"quantity": 5, "material_id": 37, "supplier_id": 6, "warehouse_id": 3}}
3	353	{"event": "material_order", "details": {"quantity": 5, "material_id": 10, "supplier_id": 5, "warehouse_id": 2}}
4	449	{"event": "material_order", "details": {"quantity": 5, "material_id": 9, "supplier_id": 6, "warehouse_id": 4}}
5	493	{"event": "material_order", "details": {"quantity": 5, "material_id": 9, "supplier_id": 1, "warehouse_id": 4}}

Рисунок 20 — Поиск заказов с количеством деталей 5

Запрос для агрегации данных (среднее значение) представлен в Листинге 6.

Листинг 6 — Агрегация по среднему

```
-- 3. Агрегация данных (среднее количество материалов в заказе)
SELECT
    AVG((event_data -> 'details' ->> 'quantity')::NUMERIC)::NUMERIC(10,2) AS
    avg_quantity
FROM borgachev_tm.user_logs
WHERE event_data ->> 'event' = 'material_order';
```

Результат запроса представлен на Рисунке 21.



Результат 1	
SELECT AVG((event_data -> 'details' ->> 'quantity')::NUMERIC)::NUMERIC(10,2) AS avg_quantity	
Таблица	avg_quantity
1	6

Рисунок 21 — Среднее значение продаж

Задание №5: модификация данных JSONB.

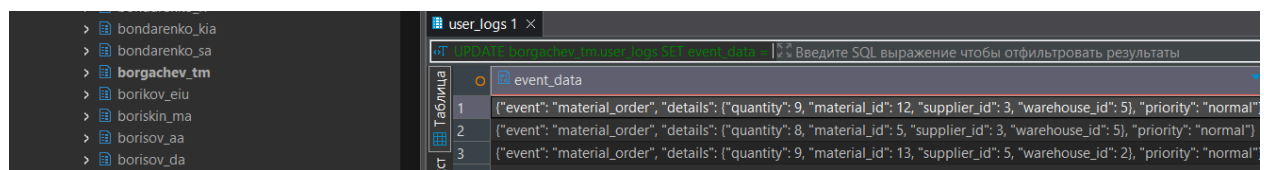
Скрипт для добавления приоритета заказам представлен в Листинге 7.

Листинг 7 — Массовое обновление полей

```
-- Массовое добавление поля "priority" для всех заказов материалов
UPDATE borgachev_tm.user_logs
SET event_data = event_data || '{"priority": "normal"}'::jsonb
WHERE event_data ->> 'event' = 'material_order';

-- Проверка
SELECT event_data
FROM borgachev_tm.user_logs
WHERE event_data ? 'priority'
LIMIT 3;
```

Результат представлен на Рисунке 22.



user_logs 1	
UPDATE borgachev_tm.user_logs SET event_data = event_data '{"priority": "normal"}'::jsonb WHERE event_data ->> 'event' = 'material_order';	
Таблица	event_data
1	{"event": "material_order", "details": {"quantity": 9, "material_id": 12, "supplier_id": 3, "warehouse_id": 5}, "priority": "normal"}
2	{"event": "material_order", "details": {"quantity": 8, "material_id": 5, "supplier_id": 3, "warehouse_id": 5}, "priority": "normal"}
3	{"event": "material_order", "details": {"quantity": 9, "material_id": 13, "supplier_id": 5, "warehouse_id": 2}, "priority": "normal"}

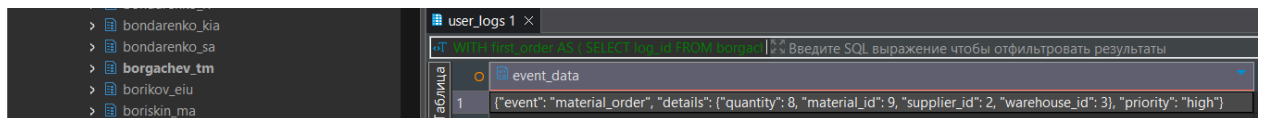
Рисунок 22 — Результат добавления приоритета заказам

Листинг 8 — Точечное изменение значения

```
-- Точечное изменение значения: меняем приоритет на "high" для конкретного заказа
WITH first_order AS (
    SELECT log_id
    FROM borgachev_tm.user_logs
    WHERE event_data ->> 'event' = 'material_order'
    ORDER BY log_id
    LIMIT 1
)
UPDATE borgachev_tm.user_logs
SET event_data = jsonb_set(event_data, '{priority}', '"high"::jsonb')
WHERE log_id = (SELECT log_id FROM first_order);

-- Проверка
SELECT event_data
FROM borgachev_tm.user_logs
WHERE event_data ->> 'priority' = 'high'
LIMIT 1;
```

Скрипт для точечного изменения значения представлен в Листинге 8.



user_logs 1 ×	
Введите SQL выражение чтобы отфильтровать результаты	
event_data	
1	{ "event": "material_order", "details": { "quantity": 8, "material_id": 9, "supplier_id": 2, "warehouse_id": 3 }, "priority": "high" }

Рисунок 23 — Результат изменения первого заказа